Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

Звіт про виконання лабораторної роботи №2
З дисципліни **«Функціональне програмування»**
На тему **«Функції вищого порядку, lambda, композиція»**
Варіант 9

Виконав:
Студент групи ФЕП-23
Литвинюк Ярослав

Перевірив:
проф. Свелеба С. А.

Львів 2026

Мета роботи: побудувати конвеєр перетворень над списком записів без мутацій.

Завдання(варіант 9): Працівники компінії.

- **Структура запису:** Працівники: id, name, department, monthly_hours.
- **конвеєр перетворень:** Обчислити сумарні години, застосувати коефіцієнт відділу, відфільтрувати за мінімумом, сформувати Top-N.
- **Reduce-агрегація:** count, total_hours, avg_hours.

Хід роботи

sample_data.json

sample_data.json

```json
[
  {
    "id": 1,
    "name": "  ivan petrenko ",
    "department": "IT",
    "monthly_hours": [80.0, 85.0]
  },
  {
    "id": 2,
    "name": "anna kovalchuk",
    "department": "HR",
    "monthly_hours": [40.0, 30.0]
  },
  {
    "id": 3,
    "name": "olena melnyk",
    "department": "Engineering",
    "monthly_hours": [90.0, 95.0]
  },
  {
    "id": 4,
    "name": "taras shevchenko",
    "department": "IT",
    "monthly_hours": [75.0, 85.0]
  },
  {
    "id": 5,
    "name": "oksana boyko",
    "department": "Engineering",
    "monthly_hours": [90.0, 95.0]
  },
  {
    "id": 6,
    "name": "dmytro bondar",
    "department": "Marketing",
    "monthly_hours": [82.0, 78.0]
  }
]
```

core.py

core.py > build_pipeline

```python
"""
core.py — Чисте функціональне ядро для Варіанта №9: Працівники компанії.
Побудова конвеєра обробки на базі map, filter, reduce та функціональної композиції.
"""

from __future__ import annotations
from functools import reduce
from itertools import islice
from typing import Any, Callable, Dict, Iterable, Iterator, List, TypedDict, TypeVar

# Структура запису згідно з варіантом №9
class EmployeeRecord(TypedDict, total=False):
    id: int
    name: str
    department: str
    monthly_hours: List[float]
    total_hours: float  # додається в результаті чистих трансформацій


T = TypeVar("T")
U = TypeVar("U")
V = TypeVar("V")


# --- Функціональні комбінатори ---

def compose(*funcs: Callable[[Any], Any]) -> Callable[[Any], Any]:
    """Математична композиція функцій: compose(f, g, h)(x) == f(g(h(x)))."""
    def _c(x: Any) -> Any:
        for f in reversed(funcs):
            x = f(x)
        return x
    return _c


def pipe(data: Any, *funcs: Callable[[Any], Any]) -> Any:
    """Прямий конвеєр трансформацій зліва направо: pipe(x, f, g) == g(f(x))."""
    acc = data
    for f in funcs:
        acc = f(acc)
    return acc

```

```python
def juxt(*funcs: Callable[[Any], Any]) -> Callable[[Any], List[Any]]:
    """Повертає функцію, яка застосовує кожну функцію до одного й того ж аргументу."""
    return lambda x: [f(x) for f in funcs]


# --- Чисті трансформації (Pure Transformations) ---

def calculate_total_hours() -> Callable[[Iterable[EmployeeRecord]], Iterator[EmployeeRecord]]:
    """Трансформація (map): обчислює сумарні години без мутації вихідних даних."""
    return lambda recs: (
        {**r, "total_hours": round(float(sum(r.get("monthly_hours", []) or [])), 2)}
        for r in recs
    )
```

```python
def boost_department_hours(
    department: str, factor: float
) -> Callable[[Iterable[EmployeeRecord]], Iterator[EmployeeRecord]]:
    """Трансформація (map): підсилює години працівників обраного відділу на заданий коефіцієнт."""
    def _boost(recs: Iterable[EmployeeRecord]) -> Iterator[EmployeeRecord]:
        for r in recs:
            curr = float(r.get("total_hours", 0.0))
            if r.get("department", "").strip().lower() == department.strip().lower():
                yield {**r, "total_hours": round(curr * factor, 2)}
            else:
                yield {**r, "total_hours": curr}
    return _boost


def filter_min_hours(
    min_hours: float = 80.0,
) -> Callable[[Iterable[EmployeeRecord]], Iterator[EmployeeRecord]]:
    """Трансформація (filter): відсіює працівників із годинами нижче мінімального порогу."""
    return lambda recs: (
        r for r in recs if float(r.get("total_hours", 0.0)) >= min_hours
    )


def normalize_names() -> Callable[[Iterable[EmployeeRecord]], Iterator[EmployeeRecord]]:
    """Трансформація (map): форматує імена у Title Case без зайвих пробілів."""
    return lambda recs: (
        {**r, "name": str(r.get("name", "")).strip().title()}
        for r in recs
    )


def sort_by_hours_desc() -> Callable[[Iterable[EmployeeRecord]], List[EmployeeRecord]]:
    """Сортування за 2 ключами: години спадно (-total_hours), тай-брейкер id зростаюче."""
    return lambda recs: sorted(
        recs,
        key=lambda r: (-float(r.get("total_hours", 0.0)), r.get("id", 0))
    )


def take(n: int) -> Callable[[Iterable[EmployeeRecord]], List[EmployeeRecord]]:
    """Зріз перших N елементів через islice (Top-N)."""
    return lambda recs: list(islice(recs, n))
```

```python
# --- reduce-агрегація ---

def reduce_stats(recs: Iterable[EmployeeRecord]) -> Dict[str, float]:
    """
    Агрегація за допомогою functools.reduce:
    обчислює count, total_hours та avg_hours.
    """
    init = {"count": 0.0, "total_hours": 0.0}
    acc = reduce(
        lambda a, r: {
            "count": a["count"] + 1,
            "total_hours": round(a["total_hours"] + float(r.get("total_hours", 0.0)), 2),
        },
        recs,
        init,
    )
    avg = round(acc["total_hours"] / acc["count"], 2) if acc["count"] > 0 else 0.0
    return {
        "count": acc["count"],
        "total_hours": acc["total_hours"],
        "avg_hours": avg,
    }
# --- Складання конвеєра через композицію ---

def build_pipeline(
    top_n: int = 3,
    department: str = "IT",
    factor: float = 1.15,
    min_hours: float = 80.0,
) -> Callable[[Iterable[EmployeeRecord]], List[EmployeeRecord]]:
    """
    Будує чистий композиційний конвеєр перетворень:
    normalize -> calculate -> boost -> filter -> sort (2 keys) -> take(Top-N).
    """
    return compose(
        take(top_n),
        sort_by_hours_desc(),
        filter_min_hours(min_hours),
        boost_department_hours(department, factor),
        calculate_total_hours(),
        normalize_names(),
    )
```

app.py

app.py > load_data

```python
"""app.py — CLI точка входу (Imperative Shell).

Відповідає за зчитування аргументів командного рядка, завантаження JSON
файлу та виведення звіту в консоль.
"""

import argparse
import json
import os
from typing import Any, Dict, List

from core import EmployeeRecord, build_pipeline, juxt, reduce_stats

def load_data(file_path: str) -> List[EmployeeRecord]:
    """Зчитує вхідні дані з JSON файлу або повертає тестовий набір за замовчуванням."""
    if os.path.exists(file_path):
        with open(file_path, "r", encoding="utf-8") as f:
            data: List[EmployeeRecord] = json.load(f)
            return data

    return [
        {
            "id": 1,
            "name": "ivan petrenko",
            "department": "IT",
            "monthly_hours": [80.0, 85.0],
        },
        {
            "id": 2,
            "name": "anna kovalchuk",
            "department": "HR",
            "monthly_hours": [40.0, 30.0],
        },
        {
            "id": 3,
            "name": "olena melnyk",
            "department": "Engineering",
            "monthly_hours": [90.0, 95.0],
        },
        {
            "id": 4,
            "name": "taras shevchenko",
            "department": "IT",
            "monthly_hours": [75.0, 85.0],
        },
        {
            "id": 5,
            "name": "oksana boyko",
            "department": "Engineering",
            "monthly_hours": [90.0, 95.0],
        },
    ]

```

```python
def render_report(
    top_employees: List[EmployeeRecord], stats: Dict[str, float]
) -> None:
    """Виводить відформатовану таблицю рейтингу та підсумкову статистику в консоль."""
    print("=" * 66)
    print("      РЕЙТИНГ ПРАЦІВНИКІВ КОМПАНІЇ (ВАРІАНТ №9: TOP-N)      ")
    print("=" * 66)
    print(
        f"{'Ранг':<5} | {'ID':<4} | {'Ім'я':<18} | {'Відділ':<14} | {'Години':<8}"
    )
    print("-" * 66)

    for rank, emp in enumerate(top_employees, start=1):
        print(
            f"{rank:<5} | "
            f"{emp['id']:<4} | "
            f"{emp['name']:<18} | "
            f"{emp['department']:<14} | "
            f"{emp.get('total_hours', 0.0):<8.2f}"
        )

    print("-" * 66)
    # Застосування комбінатора juxt для вилучення значень
    get_metrics = juxt(
        lambda s: int(s["count"]),
        lambda s: s["total_hours"],
        lambda s: s["avg_hours"],
    )
    count, total_h, avg_h = get_metrics(stats)

    print("РЕЗУЛЬТАТИ REDUCE-АГРЕГАЦІЇ:")
    print(f"• Кількість у Top-N (count):      {count}")
    print(f"• Сумарні години (total_hours):   {total_h:.2f} год.")
    print(f"• Середні години (avg_hours):     {avg_h:.2f} год.")
    print("=" * 66)
```

```python
def main() -> None:
    parser = argparse.ArgumentParser(
        description="Конвеєр обробки годин працівників компанії (Функціональне програмування)"
    )
    parser.add_argument(
        "--data",
        type=str,
        default="sample_data.json",
        help="Шлях до файлу JSON",
    )
    parser.add_argument(
        "--top", type=int, default=3, help="Кількість працівників у Top-N"
    )
    parser.add_argument(
        "--boost-dept",
        type=str,
        default="IT",
        help="Відділ для застосування коефіцієнта",
    )
```

```
    parser.add_argument(
        "--factor",
        type=float,
        default=1.15,
        help="Коефіцієнт підсилення годин",
    )
    parser.add_argument(
        "--min-hours",
        type=float,
        default=80.0,
        help="Мінімальний поріг відпрацьованих годин",
    )

    args = parser.parse_args()

    records = load_data(args.data)
    pipeline = build_pipeline(
        top_n=args.top,
        department=args.boost_dept,
        factor=args.factor,
        min_hours=args.min_hours,
    )

    processed_top = pipeline(records)
    stats = reduce_stats(processed_top)

    render_report(processed_top, stats)


if __name__ == "__main__":
    main()
```

висоновок: Побудовано незмінний конвеєр обробки даних за допомогою математичної композиції `compose`, функцій вищого порядку та генераторних виразів. Реалізовано багатокритеріальне сортування для вибірки Top-N та статистичну згортку результатів через `functools.reduce` без проміжної мутації вхідних записів.